GA4 Attribution Models — Usage Guide

A comprehensive guide for marketing analysts and data engineers who want to set up, run, and extend the GA4 Attribution Models Dataform pipeline.

Version: 2.0.0
Last updated: 2026-05-05
Target audience: Marketing analysts, data engineers, BI developers with basic command-line experience
License: MIT

Table of Contents

---

1. Overview

What This Project Does

The GA4 Attribution Models project applies **eight distinct attribution models** to Google Analytics 4 session-to-conversion journeys, running entirely within Google BigQuery via Dataform. It uses Google's public GA4 sample dataset by default, so you can explore multi-touch attribution with zero data engineering — only a free Google Cloud Platform (GCP) account is required.

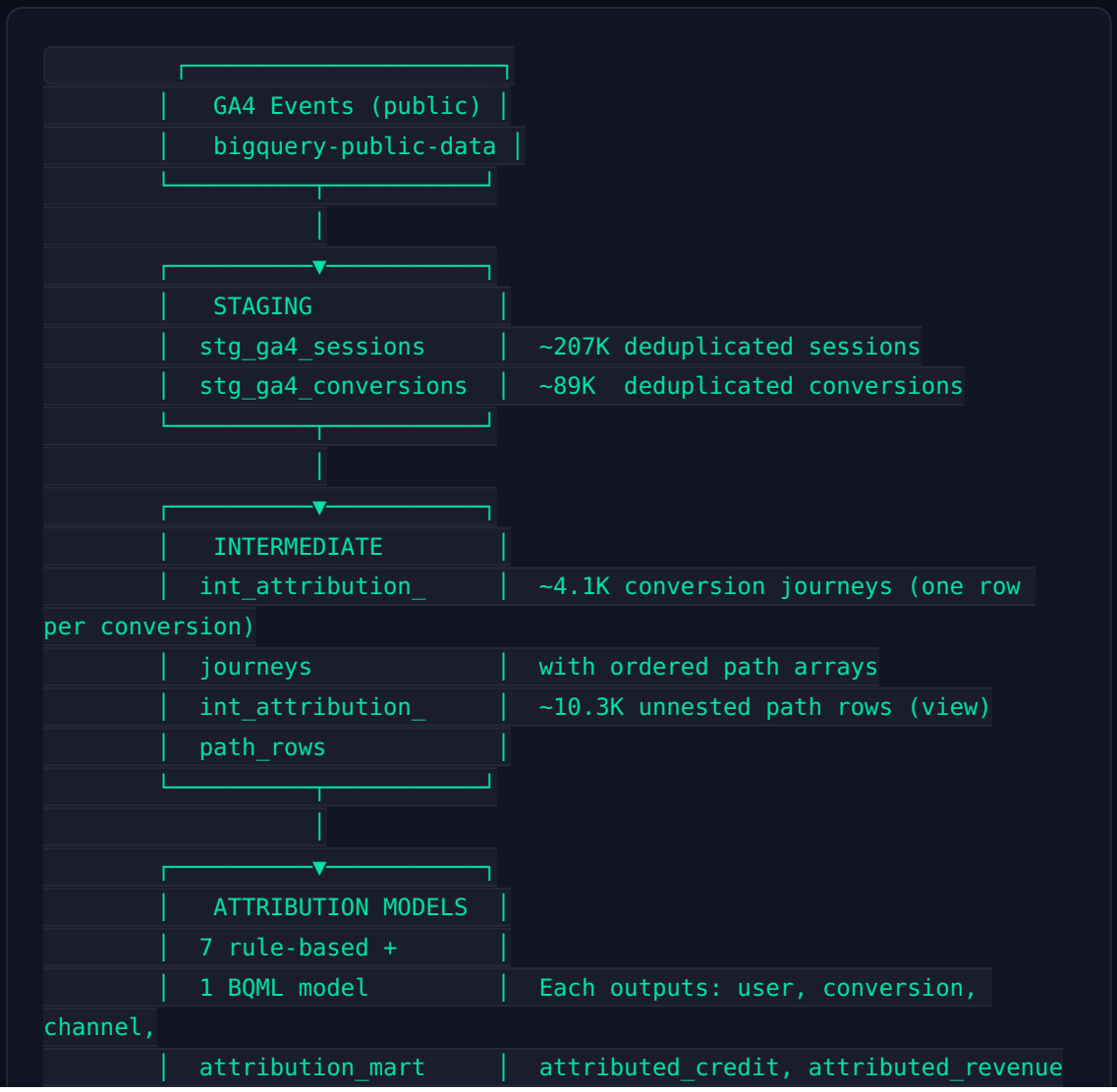
The Eight Models

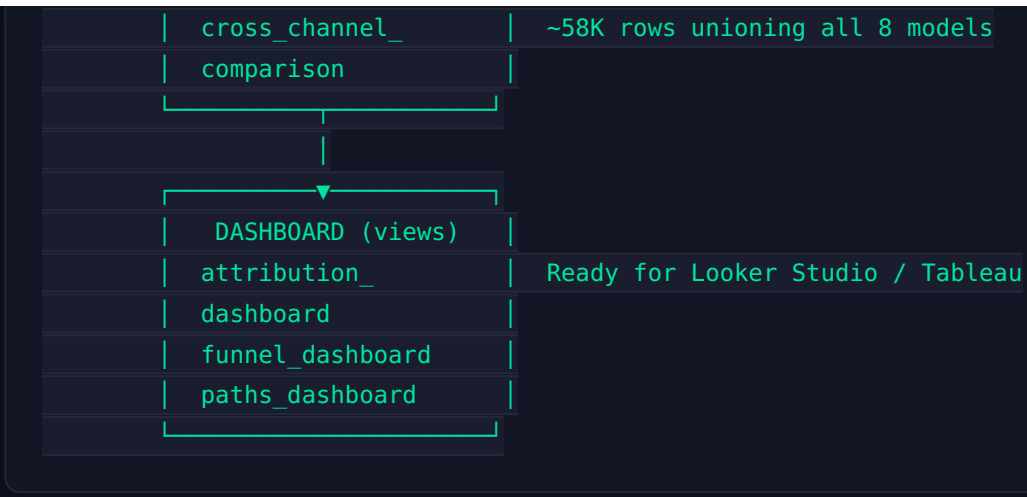
Model	Strategy	Best For
First Click	100% credit to the first session	Understanding top-of-funnel acquisition channels
Last Click	100% credit to the last session before conversion	Default reporting; identifying closing channels
Last Non-Direct Click	100% credit to the last non-Direct session; falls back to Direct if the entire path is Direct	Removing brand/ direct bias from last-click analysis
Linear	Equal credit split across all sessions	A balanced, baseline view
Time Decay	Exponential decay with a 7-day half-life; closer sessions get more credit	Emphasising recency while still valuing earlier touchpoints
U-Shaped	40% first + 40% last + 20% middle (split equally)	Valuing both discovery and closing channels

Model	Strategy	Best For
Position Weighted	Calibrated heuristic: 50% first, 30% last, 20% middle (split equally)	A stronger emphasis on acquisition than the U-shaped model
Data-Driven (BQML)	Feature-ablation attribution via BigQuery ML logistic regression; measures the marginal contribution of each channel by predicting with and without it	Statistically grounded, channel-incrementality measurement

Architecture

The pipeline follows a classic ELT (Extract, Load, Transform) pattern organised into four layers:





Key design decisions:

- **Session-based**, not event-based. Each touchpoint is a GA4 session, extracted using the GA4-UI-style "first non-auto event" rule for source/medium resolution. This aligns with the GA4 UI Session Acquisition report and reduces drift by 5–15% on production data compared to simpler extraction methods.
- **30-day lookback window** before each conversion (configurable).
- **Deduplication at every layer**: sessions, conversions, and path rows are all deduplicated with `ROW_NUMBER()` window functions.
- **Full ecommerce payload** preserved: `purchase_revenue`, `purchase_revenue_in_usd`, `transaction_id`, `items`, shipping, tax, and device/geo context.
- **Eight-channel normalisation** via a single source-of-truth function in `includes/channel_grouping.js`.
- **Multi-conversion support**: repeat purchasers get separate, independently ordered journeys.

Project File Structure

```
ga4-attribution-models/
├── workflow_settings.yaml          # Central configuration (project,
dates, dataset)
├── .df-credentials.json            # Git-ignored BigQuery credentials
├── definitions/
│   ├── sources/
│   │   └── ga4_events.sqlx         # External GA4 table declaration
│   ├── staging/
```

```
|   |   ├── stg_ga4_sessions.sqlx   # Deduplicated sessions with source/
medium
|   |   └── stg_ga4_conversions.sqlx# Deduplicated conversions with
ecommerce data
|   ├── intermediate/
|   |   ├── int_attribution_journeys.sqlx  # One row per conversion,
path array
|   |   └── int_attribution_path_rows.sqlx # Unnested paths for model
consumption
|   ├── attribution_models/
|   |   ├── attr_first_click.sqlx
|   |   ├── attr_last_click.sqlx
|   |   ├── attr_last_non_direct_click.sqlx
|   |   ├── attr_linear.sqlx
|   |   ├── attr_time_decay.sqlx
|   |   ├── attr_u_shape.sqlx
|   |   ├── attr_position_weighted.sqlx
|   |   ├── attr_data_driven_bqml.sqlx     # BQML feature-ablation
predictions
|   |   ├── attribution_mart.sqlx          # UNION ALL of all 8 models
|   |   └── cross_channel_comparison.sqlx  # Channel-level aggregation
|   ├── ecommerce_funnel/
|   |   ├── purchase_funnel.sqlx
|   |   └── cart_abandonment.sqlx
|   ├── user_journey/
|   |   └── path_analysis.sqlx
|   ├── dashboard/
|   |   ├── attribution_dashboard.sqlx
|   |   ├── funnel_dashboard.sqlx
|   |   └── paths_dashboard.sqlx
|   └── ml/
|       └── attr_data_driven_train.sqlx    # BQML logistic regression
training
├── includes/
|   ├── channel_grouping.js        # DRY 8-channel CASE logic
|   ├── source_resolution.js       # DRY GA4 UI-style source resolution
|   └── constants.js               # Safe defaults for workflow
variables
├── README.md
├── CHANGELOG.md
└── .gitignore
```

2. Prerequisites

Required Accounts & Access

1. **Google Cloud Platform (GCP) account** — A free-tier account is sufficient. You need BigQuery and the ability to create service accounts. Sign up at cloud.google.com.
2. **BigQuery enabled** — BigQuery is enabled by default on new GCP projects. Verify in the GCP Console under **APIs & Services > Enabled APIs & Services** that the BigQuery API is listed.
3. **A dedicated GCP project** — We recommend creating a project specifically for this pipeline (e.g., `marketing-attribution`). Your project ID appears in `workflow_settings.yaml` and `.df-credentials.json`.

Service Account

Create a service account with the following BigQuery roles:

- **BigQuery Data Editor** (`roles/bigquery.dataEditor`) — Required to create tables, views, and models in your project.
- **BigQuery User** (`roles/bigquery.user`) — Required to run queries and jobs.

Steps to create the service account:

1. In the GCP Console, navigate to **IAM & Admin > Service Accounts**.
2. Click **Create Service Account**.
3. Give it a name (e.g., `dataform-runner`) and description.
4. Grant the two roles: **BigQuery Data Editor** and **BigQuery User**.
5. Click **Done**, then click on the new service account, go to the **Keys** tab, and **Add Key > Create New Key**.
6. Choose **JSON** format and download the key file.

Important: Keep this key file secure. It will be embedded in your `.df-credentials.json` file (which is git-ignored). Never commit it to version control.

Software

Tool	Minimum Version	Installation
Node.js	18.x or later	nodejs.org or `nvm install 18`
npm	9.x or later	Bundled with Node.js
Dataform CLI	3.0.20+	`npm install -g @dataform/cli`
Git	Any recent version	git-scm.com

Verify your installation:

```
node --version    # Should be v18.x or later
npm --version     # Should be 9.x or later
dataform --version  # Should show v3.x
git --version
```

BigQuery Dataset for the Public Sample

The public GA4 dataset (`bigquery-public-data.ga4_obfuscated_sample_ecommerce`) is a US multi-region dataset. Your GCP project's BigQuery queries must run in a **US** location to read from it without cross-region charges. The default `location: US` in `workflow_settings.yaml` handles this.

---

3. Step-by-Step Setup

3.1 Clone the Repository

```
git clone https://github.com/GlitchG/ga4-attribution-models.git
cd ga4-attribution-models
```

3.2 Create the Credentials File

Create `.df-credentials.json` in the project root. This file is git-ignored (listed in `.gitignore`) so it won't be accidentally committed.

The format for Dataform CLI v3 is:

```
{
  "projectId": "YOUR_GCP_PROJECT_ID",
  "location": "US",
  "credentials": "<escaped service account JSON string>"
}
```

How to create the `credentials` value:

The `credentials` field must be the **entire content of your service account JSON key file**, properly escaped as a single-line string. Here's the easiest method:

```
# From your project root, read the key file and write the credentials
file in one command:
node -e "
const sa = JSON.stringify(JSON.stringify(require('fs').readFileSync('/
path/to/your-service-account-key.json', 'utf8')));
console.log(JSON.stringify({
  projectId: 'YOUR_GCP_PROJECT_ID',
  location: 'US',
  credentials: JSON.parse(sa)
}, null, 2));
" > .df-credentials.json
```

Or manually, by copying the entire JSON content of your service account key and replacing all newlines with `\n` and all double-

quotes with `\"`. Then wrap it in quotes as the value of the `credentials` field.

Example structure (with truncated key):

```
{
  "projectId": "my-attribution-project",
  "location": "US",
  "credentials": "{\n  \"type\": \"service_account\",\n  \"project_id\": \"my-attribution-project\",\n  \"private_key_id\": \"abc123...\",\n  \"private_key\": \"-----BEGIN PRIVATE KEY-----\\nMIIEv...\\n-----END PRIVATE KEY-----\\n\",\n  \"client_email\": \"dataform-runner@my-attribution-project.iam.gserviceaccount.com\"\n}"
}
```

Testing credentials — after creating the file, verify that Dataform can authenticate:

```
dataform init-creds .df-credentials.json
```

If successful, you'll see a confirmation message. If not, double-check that the `credentials` field is properly escaped JSON (not the raw JSON object).

3.3 Configure `workflow_settings.yaml`

Edit the file to match your GCP environment:

```
dataformCoreVersion: "3.0.20"
defaultProject: YOUR_GCP_PROJECT_ID        # ← Change this
defaultLocation: US                        # Must be US for the public dataset
defaultDataset: attribution_models         # Output dataset for all tables
defaultAssertionDataset: attribution_assertions  # Dataset for data quality assertions
vars:
  start_date: "20201101"                   # Public dataset range
  end_date: "20201220"                     # Public dataset range
  ga4_project: "bigquery-public-data"      # Source GA4 project
  ga4_dataset: "ga4_obfuscated_sample_ecommerce"  # Source GA4 dataset
```

```
  lookback_days: "30"                       # Attribution window in
days
```

Key configuration fields:

Variable	Purpose	Default
`defaultProject`	Your GCP project where output tables are created	Must be changed
`defaultLocation`	BigQuery region for queries and storage	`US`
`defaultDataset`	Schema/dataset name for output tables	`attribution_models`
`start_date` / `end_date`	Date range filter (`_TABLE_SUFFIX`)	`20201101` – `20201220`
`ga4_project` / `ga4_dataset`	Source GA4 data location	Public dataset
`lookback_days`	How far before a conversion to include sessions	`30`

Important: The `defaultProject` **must match** the `projectId` in your `.df-credentials.json` . The source GA4 dataset can be in a different project (the public dataset lives in `bigquery-public-data`).

3.4 Install Dependencies

The project has no `package.json` — Dataform CLI is your only dependency. Install it globally:

```
npm install -g @dataform/cli@^3.0.20
```

Verify with:

```
dataform --version
# Should output: 3.x.x
```

3.5 Compile to Verify

Before running the full pipeline, compile the project to check for syntax errors and validate the dependency graph:

```
dataform compile --default-database=YOUR_GCP_PROJECT_ID
```

You should see output listing each compiled action (tables, views, operations, assertions) with a green checkmark. If compilation fails, see the Common Issues section.

---

4. Running the Pipeline

4.1 First Run (Full Refresh)

For the first execution, use `--full-refresh` to create all tables from scratch:

```
dataform run --default-database=YOUR_GCP_PROJECT_ID --full-refresh
```

What happens during a full run:

1. **Staging** (`stg_ga4_sessions`, `stg_ga4_conversions`) — Extracts sessions and conversions from the public GA4 dataset with deduplication.
2. **Intermediate** (`int_attribution_journeys`, `int_attribution_path_rows`) — Builds conversion journeys with ordered path arrays, joined within the 30-day lookback window.
3. **Attribution Models** — Each of the 8 models runs independently, reading from the intermediate layer.
4. **Mart** (`attribution_mart`) — Unions all 8 model outputs into a single table.

5. **Cross-Channel Comparison** — Aggregates channel-level metrics across all models.
6. **Dashboard Views** — Creates three dashboard-ready views.
7. **BQML Training** — Trains the logistic regression model (`ml/attr_data_driven_train` operation), then runs predictions (`attr_data_driven_bqml`).

Expected runtime: 5–15 minutes on the public dataset (depends on BigQuery slot availability). The BQML training step adds 1–3 minutes.

Expected output row counts (public dataset, 2020-11-01 to 2020-12-20):

Table	Approximate Rows
`stg_ga4_sessions`	207,000
`stg_ga4_conversions`	89,000
`int_attribution_journeys`	4,100
`int_attribution_path_rows`	10,300
`attribution_mart`	58,000 (all 8 models combined)
`attr_data_driven_bqml`	4,900 (may produce 0 rows if training data is too sparse)

4.2 Selective Runs with Tags

Once the full pipeline has been built, you can run subsets of the DAG using tags:

```
 # Run only staging tables
dataform run --default-database=YOUR_GCP_PROJECT_ID --tags staging

# Run only attribution models
dataform run --default-database=YOUR_GCP_PROJECT_ID --tags attribution

# Run only models (not staging or intermediate)
dataform run --default-database=YOUR_GCP_PROJECT_ID --tags model
```

```
# Run only dashboard views
dataform run --default-database=YOUR_GCP_PROJECT_ID --tags dashboard

# Run the ML pipeline (training + predictions)
dataform run --default-database=YOUR_GCP_PROJECT_ID --tags ml

# Run the daily refresh subset (staging, intermediate, mart, dashboards)
dataform run --default-database=YOUR_GCP_PROJECT_ID --tags daily
```

Available tags:

Tag	What It Covers
`staging`	`stg_ga4_sessions`, `stg_ga4_conversions`
`intermediate`	`int_attribution_journeys`, `int_attribution_path_rows`
`attribution`	All attribution model tables + BQML predictions
`model`	Only the rule-based model tables (excludes BQML)
`ml`	BQML training + predictions
`mart`	`attribution_mart`, `cross_channel_comparison`
`dashboard`	`attribution_dashboard`, `funnel_dashboard`, `paths_dashboard`
`funnel`	`purchase_funnel`, `cart_abandonment`
`journey`	`path_analysis`
`daily`	`staging` + `intermediate` + `mart` + `dashboard` (intended for incremental daily refreshes)

4.3 Running Individual Actions

You can run a single table or view by its fully qualified name:

```
dataform run --default-database=YOUR_GCP_PROJECT_ID --actions staging.stg_ga4_sessions
```

```
dataform run --default-database=YOUR_GCP_PROJECT_ID --actions
attribution_models.attr_first_click
```

Dataform automatically includes all upstream dependencies when you specify an action. To run an action without dependencies:

```
dataform run --default-database=YOUR_GCP_PROJECT_ID --actions
attribution_models.attr_first_click --no-dependencies
```

4.4 Viewing the Dependency Graph

To visualise the pipeline DAG:

```
dataform compile --default-database=YOUR_GCP_PROJECT_ID --json >
dag.json
```

This produces a JSON representation of every action and its dependencies, which you can use with visualisation tools or simply inspect to understand the pipeline flow.

4.5 Running Assertions

Assertions validate data quality at each layer. They run automatically as part of the pipeline. To run only assertions:

```
dataform run --default-database=YOUR_GCP_PROJECT_ID --tags assertions
```

Assertions check: - **Unique keys**: no duplicate user-session or user-conversion pairs - **Non-null fields**: critical columns are not null - **Row conditions**: revenue >= 0, path_length >= 1, timestamps are valid

Failed assertions appear in the run output with red ✗ markers. View assertion results in the `attribution_assertions` dataset in BigQuery.

---

5. Understanding the Output

All output tables and views are created in your `defaultDataset` (usually `attribution_models`), with sub-datasets for each pipeline layer. Here's a detailed description of each key table.

5.1 Staging Layer

`staging.stg_ga4_sessions`

One row per unique session. Extracted from GA4 events using the **first non-auto event** rule (same logic the GA4 UI uses for Session Acquisition). Sessions are deduplicated by `(user_pseudo_id, session_id)`.

Key columns:

Column	Type	Description
`user_pseudo_id`	STRING	GA4 user identifier (cookie-level)
`session_id`	INT64	GA4 session identifier
`session_start`	TIMESTAMP	Session start time
`source`	STRING	Traffic source (e.g., `google`, `bing`, `(direct)`)
`medium`	STRING	Traffic medium (e.g., `organic`, `cpc`, `referral`)
`campaign`	STRING	UTM campaign parameter
`channel`	STRING	Normalised channel (one of 8 values)

The eight channel values: `Paid Search`, `Organic Search`, `Social`, `Email`, `Display`, `Referral`, `Affiliate`, `Direct`. Any source/medium combination not matching these rules is stored as `source / medium` for transparency.

`staging.stg_ga4_conversions`

One row per conversion event (purchase, begin_checkout, add_to_cart, add_payment_info, add_shipping_info). Deduplicated by `(user_pseudo_id, event_timestamp, event_name, transaction_id)`.

Key columns:

Column	Type	Description
`user_pseudo_id`	STRING	GA4 user identifier
`conversion_ts`	TIMESTAMP	When the conversion occurred
`conversion_event`	STRING	Event type: `purchase`, `begin_checkout`, etc.
`transaction_id`	STRING	Ecommerce transaction ID
`purchase_revenue`	FLOAT64	Revenue in local currency
`purchase_revenue_in_usd`	FLOAT64	Revenue in USD
`total_item_quantity`	INT64	Total items in the transaction
`shipping_value`	FLOAT64	Shipping cost
`tax_value`	FLOAT64	Tax amount
`device_category`	STRING	`desktop`, `mobile`, `tablet`
`country`, `region`, `city`	STRING	Geographic context

5.2 Intermediate Layer

`intermediate.int_attribution_journeys`

The heart of the pipeline. **One row per purchase conversion** with an ordered array of all sessions that occurred within the lookback window. Multi-conversion users get separate rows with independent journeys.

Key columns:

Column	Type	Description
`user_pseudo_id`	STRING	GA4 user
`conversion_id`	STRING	Globally unique conversion ID (`user_pseudo_id-seq`)
`conversion_ts`	TIMESTAMP	Conversion timestamp
`path`	ARRAY\<STRUCT>	Ordered array of session touchpoints
`path_length`	INT64	Number of sessions in the journey
`purchase_revenue_in_usd`	FLOAT64	Total revenue for this conversion

Each element in the `path` array contains: `session_start` , `source` , `medium` , `campaign` , `content` , `term` , `channel` , `hours_before_conversion` .

`intermediate.int_attribution_path_rows`

A **view** that unnests the journey paths into one row per session per conversion. This is the primary input for all attribution models.

Contains ascending and descending position numbers (`session_position_asc`, `session_position_desc`) for easy first/last session identification.

5.3 Attribution Model Tables

Each model table (e.g., `attribution_models.attr_first_click`, `attribution_models.attr_time_decay`) has identical output schema:

Column	Type	Description
`user_pseudo_id`	STRING	GA4 user
`conversion_id`	STRING	Unique conversion identifier
`conversion_ts`	TIMESTAMP	Conversion timestamp
`transaction_id`	STRING	Ecommerce transaction ID
`purchase_revenue`	FLOAT64	Revenue in local currency
`purchase_revenue_in_usd`	FLOAT64	Revenue in USD
`path_length`	INT64	Journey length
`source`	STRING	Traffic source of this touchpoint
`medium`	STRING	Traffic medium
`campaign`	STRING	UTM campaign
`channel`	STRING	Normalised channel
`model`	STRING	Model identifier (e.g., `'first_click'`)
`attributed_credit`	FLOAT64	

Column	Type	Description
		Share of conversion credit (sums to 1.0 per conversion)
`attributed_revenue`	FLOAT64	Attributed revenue in USD
`attributed_revenue_local`	FLOAT64	Attributed revenue in local currency

`attribution_models.attribution_mart`

A **union of all 8 model outputs** into a single table. This is your primary table for reporting, BI tools, and cross-model comparison. Each row is one touchpoint within one conversion under one model.

Row count: ~58,000 rows on the public dataset (4,100 conversions × ~1.8 avg path length × 8 models).

`attribution_models.cross_channel_comparison`

Channel-level aggregation across all models. Use this for high-level model comparison.

Key columns:

Column	Type	Description
`model`	STRING	Model name
`channel`	STRING	Channel
`attributed_conversions`	INT64	Distinct conversions with this channel present
`total_credit`	FLOAT64	Sum of attributed credits
`total_revenue_usd`	FLOAT64	Sum of attributed revenue (USD)

Column	Type	Description
`avg_order_value_usd`	FLOAT64	Revenue per credited conversion

5.4 BQML Data-Driven Model

`ml.attr_data_driven_model`

A BigQuery ML **logistic regression model** trained on binary channel features. The training data includes: - **Positive samples**: users who converted, with flags for each channel present in their path. - **Negative samples**: users with sessions but no conversions in the window (negative sampling to avoid degenerate training).

The model uses `auto_class_weights = TRUE` to handle class imbalance and `max_iterations = 50`.

`attribution_models.attr_data_driven_bqml`

Applies **feature ablation** to compute marginal channel contributions: 1. Predict P(conversion | all channels) for each conversion. 2. For each of the 8 channels, predict P(conversion | channel removed). 3. Removal effect = P(all) − P(without channel). 4. Normalise effects to sum to 1.0 per conversion.

Important caveats: - The model requires sufficient training data (>100 conversions recommended). On the public dataset with only 4,100 conversions across 8 channels, the model may produce zero rows or unreliable results. - This is **feature ablation**, not true Shapley values (which would require 2^8 = 256 predictions per conversion). It approximates marginal contribution in a single pass. - The model dependencies require the training operation to run before predictions. Dataform handles this via the explicit `dependencies: ["attr_data_driven_train"]` in the predictions config.

5.5 Dashboard Views

Three views are designed for direct connection to Looker Studio, Tableau, or any BI tool that supports BigQuery:

`dashboard.attribution_dashboard`

Flattened attribution model comparison. Includes per-channel metrics (`total_revenue_usd`, `total_credit`, `aov_usd`) joined from the cross-channel comparison. Use for: - Bar charts comparing model crediting by channel - Revenue attribution heatmaps - Model comparison tables

`dashboard.funnel_dashboard`

Ecommerce funnel stages (`add_to_cart` → `begin_checkout` → `add_shipping_info` → `add_payment_info` → `purchase`) with user counts, session counts, and cart abandonment rates. Use for: - Funnel visualisation widgets - Drop-off analysis

`dashboard.paths_dashboard`

User journey paths with channel sequences as readable strings (e.g., `Organic Search > Direct > Paid Search`). Includes device and geo context. Use for: - Top conversion path tables - Path pattern analysis

5.6 Additional Outputs

Table / View	Purpose
`ecommerce_funnel.purchase_funnel`	Staged ecommerce funnel with conversion rates
`ecommerce_funnel.cart_abandonment`	Users who added to cart but didn't purchase
`user_journey.path_analysis`	Top 100 conversion paths by frequency and revenue

5.7 Sample Queries

Compare model crediting for Organic Search:

```
SELECT
  model,
  ROUND(SUM(attributed_credit), 2) AS total_credit,
  ROUND(SUM(attributed_revenue), 2) AS total_revenue_usd,
  ROUND(COUNT(DISTINCT conversion_id), 0) AS conversions
FROM attribution_models.attribution_mart
WHERE channel = 'Organic Search'
GROUP BY 1
ORDER BY total_revenue_usd DESC;
```

View a specific user's journey with all model attributions:

```
SELECT
  user_pseudo_id,
  conversion_id,
  model,
  channel,
  attributed_credit,
  attributed_revenue
FROM attribution_models.attribution_mart
WHERE user_pseudo_id = '12345.67890'
ORDER BY conversion_id, model, channel;
```

Get top conversion paths:

```
SELECT * FROM dashboard.paths_dashboard
ORDER BY purchase_revenue_in_usd DESC
LIMIT 20;
```

---

6. Publishing to BigQuery

6.1 Using the Public GA4 Dataset (Default)

No setup needed. The pipeline reads from `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` by default. Simply ensure:

- Your `location` in `.df-credentials.json` and `workflow_settings.yaml` is `US`
- `ga4_project` is `bigquery-public-data`
- `ga4_dataset` is `ga4_obfuscated_sample_ecommerce`

The public dataset covers **November 1 – December 20, 2020** and contains obfuscated (anonymised) data from the Google Merchandise Store. It's sufficient for learning and testing all eight models.

Note on query costs: Reading from the public dataset is **free** (you're not charged for storage), but you pay for the BigQuery compute (analysis) costs of your queries. On the free tier, the first 1 TB of queries per month is free. The full pipeline run processes less than 10 GB — well within the free tier.

6.2 Using Your Own GA4 BigQuery Export

To run the pipeline against your own GA4 data, update three variables in `workflow_settings.yaml`:

```
vars:
  start_date: "20240101"           # Match your GA4 export date range
  end_date: "20241231"
  ga4_project: "my-ga4-project"    # Your GCP project hosting the GA4 export
  ga4_dataset: "analytics_123456789"  # Your GA4 dataset ID
  lookback_days: "30"
```

No SQL changes needed. The pipeline uses `${constants.GA4_PROJECT}`, `${constants.GA4_DATASET}`, `${constants.START_DATE}`, and `${constants.END_DATE}` throughout, which resolve from the workflow variables.

Important considerations for your own data:

1. **Source/medium field selection** — The public dataset uses `event_params` extraction with the first-non-auto-event rule. For your own GA4 export:
2. If your export is **after mid-2024**, consider modifying `stg_ga4_sessions.sqlx` to use `session_traffic_source_last_click.cross_channel_campaign.source` / `.medium` instead. This field resolves the google/cpc misattribution bug where Google Ads clicks without UTM parameters appear as `google / organic`.
3. If your export is **after June 2023**, you can use `collected_traffic_source.manual_source` / `.manual_medium`.
4. The current `event_params` approach works on all export dates.
5. **Never** use `traffic_source.source` / `.medium` — these are user-level first-touch values that persist across all sessions, making every attribution model produce identical results.
6. **Date sharding** — The pipeline uses `_TABLE_SUFFIX BETWEEN start_date AND end_date` to filter event tables. If your GA4 export uses a different sharding pattern, you may need to adjust the source declaration in `definitions/sources/ga4_events.sqlx`.
7. **Lookback window truncation** — When dates are used as `_TABLE_SUFFIX` filters, the 30-day lookback cannot extend before `start_date`. Sessions in early December 2020 that occurred in late November are excluded because the filter starts at 2020-11-01. See Common Issues for details.
8. **Cross-project access** — If your GA4 export is in a different GCP project than your attribution pipeline, ensure your service account has **BigQuery Data Viewer** (`roles/bigquery.dataViewer`) on the source project.
9. **Scale considerations** — The pipeline uses `CREATE TABLE AS SELECT` for all tables (no partitioning). On datasets with millions of sessions, this is still efficient since BigQuery's columnar storage handles wide scans well. For very large datasets (billions of events), consider:
10. Narrower date ranges

11. Pre-filtering events before the pipeline run
12. Talking to your data engineering team about adding date partitioning to the source tables

6.3 Connecting BI Tools

After the pipeline runs, connect your BI tool to BigQuery:

Looker Studio: 1. Create a new data source → BigQuery connector. 2. Select your project and the `dashboard` dataset. 3. Choose `attribution_dashboard` as your table. 4. Build charts: model × channel bar charts, funnel widgets, path tables.

Tableau: 1. Connect to BigQuery using the Tableau BigQuery connector. 2. Select your project → `attribution_models` → `attribution_mart`. 3. Use `model` and `channel` as dimensions, `SUM(attributed_revenue)` as the measure.

Google Sheets (Connected Sheets): 1. Open Sheets → Data → Data Connectors → BigQuery. 2. Connect to `attribution_models.cross_channel_comparison`. 3. Build pivot tables for model comparison.

---

7. Common Issues & Troubleshooting FAQ

7.1 Authentication & Credentials

Q: `dataform run` fails with "Could not authenticate" or "invalid_grant"?

A: This usually means your `.df-credentials.json` is malformed. Common fixes: - Ensure the `credentials` value is a **string** containing escaped JSON, not a nested JSON object. The entire file should be a flat JSON object with three keys. - Run `dataform init-creds .df-credentials.json` to validate the format. - Ensure the service account has **BigQuery Data Editor** and **BigQuery User** roles. - If using `gcloud auth application-default login`, note that Dataform CLI requires the `.df-credentials.json` format specifically

— it does not read `~/.config/gcloud/application_default_credentials.json` by default.

Q: I get "Access Denied: BigQuery: Permission bigquery.tables.create denied"?

A: Your service account is missing the **BigQuery Data Editor** role. Grant it in IAM & Admin. Remember that IAM changes can take up to 2 minutes to propagate.

7.2 0-Row Tables from Partitioning Bug

Q: My staging tables have 0 rows after a successful run?

A: This was a known bug in an earlier version where `CREATE TABLE AS SELECT ... PARTITION BY ...` silently created empty tables. This has been **fixed** — all table configs now omit `partitionBy` and `clusterBy`. If you're still seeing 0 rows, ensure you're on the latest commit and try:

```
# Drop and recreate all tables
dataform run --default-database=YOUR_PROJECT_ID --full-refresh
```

7.3 Lookback Window Truncation

Q: Why do some conversions have fewer sessions in their journey than expected?

A: The `_TABLE_SUFFIX BETWEEN '20201101' AND '20201220'` filter in the staging models limits which event data is read. If a conversion occurs on 2020-11-02, its 30-day lookback extends to 2020-10-03, but the earliest data available is 2020-11-01. Sessions from October are silently excluded.

Workarounds: - Set `start_date` earlier than your analysis range (e.g., if analysing December, set `start_date` to November 1st). - For production pipelines, remove the `_TABLE_SUFFIX` filter from the staging models and instead filter by `event_timestamp` directly:

```
WHERE TIMESTAMP_MICROS(event_timestamp) >= TIMESTAMP('2020-11-01')
 AND TIMESTAMP_MICROS(event_timestamp) <  TIMESTAMP('2020-12-21')
```

This reads more data but provides an accurate lookback. The trade-off is query cost vs. attribution accuracy.

7.4 BQML Training Issues

Q: The `attr_data_driven_bqml` table is empty or the training failed?

A: Several possible causes:

1. **"Input data doesn't contain any rows"** — This means the training query returned zero rows. This was fixed in the latest version by adding negative samples (non-converted user-paths). Ensure you're on the latest commit.
2. **Too few conversions** — BQML logistic regression needs enough data to converge. With the public dataset's ~4,100 conversions spread across 8 channels, the model may not reach meaningful separation. For reliable data-driven attribution, aim for at least **500+ conversions and 1,000+ non-converting users**.
3. **Training ran before predictions** — This was also fixed by adding `dependencies: ["attr_data_driven_train"]` explicitly. If upgrading from an older version, re-run the ML pipeline:

```
dataform run --default-database=YOUR_PROJECT_ID --tags ml --full-refresh
```

1. **Model convergence failure** — Check the BigQuery ML model in the GCP Console under your project → `ml` → `attr_data_driven_model`. Look at the training info tab for convergence status, iteration count, and ROC AUC. A model that fails to converge (all coefficients near zero) indicates the channel features don't discriminate converters from non-converters — this is expected on small datasets.

Q: Can I skip the BQML model if I only want rule-based attribution?

A: Yes. The BQML model is entirely optional. Run the pipeline without it:

```
dataform run --default-database=YOUR_PROJECT_ID --tags daily,attribution
--exclude-tags ml
```

The `attribution_mart` will still have 7 models' worth of data. The `data_driven_bqml` rows will simply be absent from the union.

7.5 Compilation Errors

Q: `dataform compile` fails with "Could not resolve ref..."?

A: This happens when Dataform can't find a referenced table. Common causes: - You haven't created the tables yet (compile doesn't check data, only schema references). - A `.sqlx` file has a typo in the `ref("schema", "name")` call. - The `schema` in the config block doesn't match the directory structure convention.

Q: "vars.ga4_project is not defined"?

A: Ensure your `workflow_settings.yaml` has a `vars:` section with all required variables. The `includes/constants.js` file provides safe defaults, but if the YAML is malformed (bad indentation), the vars won't parse.

7.6 Cost Concerns

Q: How much will this cost on BigQuery?

A: For the public dataset (full pipeline, one run): - **Processed data**: ~8–12 GB total across all queries - **Free tier**: 1 TB/month free → ~80–120 full pipeline runs per month before any charges - **Beyond free tier**: ~$0.05–0.08 per full run (at $6.25/TB on-demand pricing) - **Storage**: All output tables combined ~50–150 MB → negligible cost (~$0.001/month)

For your own GA4 data, costs scale with data volume. A typical mid-size ecommerce site (1M sessions/month) processes ~200–500 GB per run.

Tips to reduce costs: - Use shorter date ranges during development - Run only the tags you need (e.g., `--tags staging` while debugging) - Schedule pipeline runs during off-peak hours for lower slot contention - Consider BigQuery flat-rate pricing if running at enterprise scale

7.7 Dependency Chain Issues

Q: Some downstream tables didn't refresh after I changed an upstream table?

A: Dataform uses `${ref()}` declarations to build the dependency graph automatically. If you've manually deleted a table or edited a file after compilation, run with `--full-refresh` to force all tables to rebuild.

Q: What's the correct run order?

A: Dataform determines this automatically, but here's the logical flow:

```
sources.ga4_events
 → staging.stg_ga4_sessions
 → staging.stg_ga4_conversions
   → intermediate.int_attribution_journeys
     → intermediate.int_attribution_path_rows
       → attribution_models.attr_* (all 7 rule-based models)
       → attribution_models.attribution_mart
         → attribution_models.cross_channel_comparison
         → dashboard.attribution_dashboard
       → ml.attr_data_driven_train
         → attribution_models.attr_data_driven_bqml
           → (also feeds into attribution_mart)
```

---

8. Next Steps

8.1 Add Custom Channel Groupings

The channel taxonomy is defined once in `includes/channel_grouping.js`. To add custom channels (e.g., separating branded vs. non-branded paid search), edit the CASE expression:

```
function channelGrouping(mediumExpr, sourceExpr) {
  return `CASE
    WHEN COALESCE(${mediumExpr}, '(none)') IN ('cpc', 'ppc',
'paidsearch')
      AND LOWER(COALESCE(${sourceExpr}, '')) LIKE '%brand%'
      THEN 'Paid Search - Brand'
    WHEN COALESCE(${mediumExpr}, '(none)') IN ('cpc', 'ppc',
'paidsearch')
      THEN 'Paid Search - Non-Brand'
    WHEN COALESCE(${mediumExpr}, '(none)') = 'organic' THEN 'Organic
Search'
    -- ... rest of cases ...
    ELSE CONCAT(COALESCE(${sourceExpr}, '(direct)'), ' / ', COALESCE($
{mediumExpr}, '(none)'))
  END`;
}

module.exports = { channelGrouping };
```

All models automatically pick up the new channels because they all call `${channel_grouping.channelGrouping(...)}`. No model SQL needs to change.

If you add or remove channels, you must also update the BQML training and prediction files: - `definitions/ml/attr_data_driven_train.sqlx` — add/remove channel flag columns - `definitions/attribution_models/attr_data_driven_bqml.sqlx` — add/remove channel prediction blocks and the channel list in the `CROSS JOIN UNNEST`

8.2 Add a New Attribution Model

Adding a new model requires only one new `.sqlx` file:

1. Create `definitions/attribution_models/attr_your_model.sqlx`.
2. Model config:

```
config {
  type: "table",
  schema: "attribution_models",
  name: "attr_your_model",
  tags: ["attribution", "model"],
  description: "Your custom model description."
}
```

1. Read from the intermediate layer and output the standard schema:

```
SELECT
  user_pseudo_id,
  conversion_id,
  conversion_ts,
  transaction_id,
  purchase_revenue,
  purchase_revenue_in_usd,
  path_length,
  source,
  medium,
  campaign,
  channel,
  'your_model_name' AS model,
  -- Your attribution logic here, e.g.:
  1.0 / path_length AS attributed_credit,
  purchase_revenue_in_usd / path_length AS attributed_revenue,
  purchase_revenue / path_length AS attributed_revenue_local
FROM ${ref("intermediate", "int_attribution_path_rows")}
```

1. Add a `UNION ALL SELECT * FROM ${ref("attribution_models", "attr_your_model")}` line to `definitions/attribution_models/attribution_mart.sqlx`.
2. Update `cross_channel_comparison.sqlx` if needed (it reads from `attribution_mart`, so it may work automatically).

8.3 Use with Your Own GA4 Data

See Section 6.2 for the basic configuration. Additional steps for production use:

Schedule automated runs:

Use Cloud Scheduler + Cloud Run or Workflows to trigger Dataform runs on a schedule. Example approach:

```
# Create a Cloud Run job that executes:
dataform run --default-database=$PROJECT_ID --tags daily
```

Set environment variables for credentials and project ID. Schedule daily after your GA4 export completes (typically 8–12 hours after midnight UTC).

Dataform Cloud alternative:

If you prefer a managed service, Dataform Cloud (now part of Google Cloud) can run this project natively. Import the repository and configure the workflow settings through the UI.

Incremental refresh strategy:

For ongoing production use with your own GA4 data:

1. Set up a daily Dataform run with `--tags daily` to refresh staging, intermediate, mart, and dashboards.
2. Run `--tags attribution --full-refresh` weekly to recalculate all models.
3. Run `--tags ml` monthly or when you have >2 weeks of new data for the BQML model.

8.4 Customise the Lookback Window

Change `lookback_days` in `workflow_settings.yaml` to adjust how far back the pipeline looks for sessions before each conversion:

```
vars:
  lookback_days: "60"  # 60-day attribution window
```

The pipeline uses this value in `int_attribution_journeys.sqlx`:

```
 AND s.session_start >= TIMESTAMP_SUB(c.conversion_ts, INTERVAL ${constants.LOOKBACK_DAYS} DAY)
```

Common lookback windows: 7 days (short cycle products), 30 days (standard ecommerce), 90 days (considered purchases like B2B or high-ticket items).

8.5 Monitor Data Quality

The pipeline includes built-in assertions. View their results:

```
 SELECT * FROM attribution_assertions.assertions_stg_ga4_sessions
UNION ALL
SELECT * FROM attribution_assertions.assertions_stg_ga4_conversions
UNION ALL
SELECT * FROM attribution_assertions.assertions_int_attribution_journeys
UNION ALL
SELECT * FROM
attribution_assertions.assertions_int_attribution_path_rows
ORDER BY assertion_name;
```

Failed assertions indicate data quality issues — investigate before trusting attribution results.

8.6 Join with Cost Data

For ROAS (Return on Ad Spend) analysis, join the attribution output with your ad platform cost data:

```
 WITH cost AS (
  SELECT 'Paid Search' AS channel, 5000.00 AS spend_usd
  UNION ALL
  SELECT 'Social', 3000.00
  -- ... more channels
)
SELECT
  m.model,
  m.channel,
  SUM(m.attributed_revenue) AS total_revenue,
  c.spend_usd,
```

```
  ROUND(SUM(m.attributed_revenue) / NULLIF(c.spend_usd, 0), 2) AS roas
FROM attribution_models.cross_channel_comparison m
LEFT JOIN cost c USING (channel)
GROUP BY 1, 2, c.spend_usd
ORDER BY roas DESC;
```

8.7 Explore Related Projects

- bigquery-meridian-mmm — Bayesian Media Mix Modeling for incrementality measurement, complementing attribution analysis.
- ga4-bigquery-incremental — Dataform-native GA4 incremental refresh pipeline for production GA4 data.
- marketing_analytics_sample_reporting — dbt project for paid ads reporting.

Quick Reference

Common Commands

```
 # Full pipeline from scratch
dataform run --default-database=PROJECT_ID --full-refresh

# Selective runs
dataform run --default-database=PROJECT_ID --tags staging
dataform run --default-database=PROJECT_ID --tags attribution
dataform run --default-database=PROJECT_ID --tags dashboard
dataform run --default-database=PROJECT_ID --tags daily

# Compile only (check for errors)
dataform compile --default-database=PROJECT_ID

# Run a single action
dataform run --default-database=PROJECT_ID --actions
staging.stg_ga4_sessions

# Run with assertions only
dataform run --default-database=PROJECT_ID --tags assertions
```

Key Files to Edit

File	What to Change
`workflow_settings.yaml`	Project ID, dates, GA4 source, lookback
`.df-credentials.json`	Service account credentials
`includes/channel_grouping.js`	Custom channel definitions
`definitions/attribution_models/attribution_mart.sqlx`	Add new model UNION ALL lines

Output Schema Summary

Dataset	Key Tables	Primary Use
`staging`	`stg_ga4_sessions`, `stg_ga4_conversions`	Source-extracted, deduplicated base tables
`intermediate`	`int_attribution_journeys`, `int_attribution_path_rows`	Conversion journeys for model consumption
`attribution_models`	`attribution_mart`, `cross_channel_comparison`, `attr_*`	All model outputs, reporting-ready
`dashboard`	`attribution_dashboard`, `funnel_dashboard`, `paths_dashboard`	BI tool views
`ml`		

Dataset	Key Tables	Primary Use
	`attr_data_driven_model`, `attr_data_driven_train`	BQML model and training data
`attribution_assertions`	`assertions_*`	Data quality check results

For questions, bug reports, or contributions, please open an issue on GitHub.